

OLES3D

Cumulative Core Review

Parts 1–4.2

3D medical image segmentation을 위한 핵심 복습자료

Tensor contract에서 시작해 U-Net, volumetric patch inference, NIfTI geometry와 orientation까지 구현 전에 반드시 설명할 수 있어야 하는 개념을 현재 OLES3D scratch notebook 결과와 연결했다.

범위	핵심 질문	완료 산출물
Part 1	Segmentation은 Tensor 관점에서 무엇인가?	logits/target contract, CE, Dice/IoU
Part 2	U-Net은 spatial 정보를 어떻게 압축-복원하는가?	2D U-Net, skip flow, tiny overfit
Part 3	3D volume을 제한된 memory로 어떻게 학습-추론하는가?	3D memory, sampling, sliding window
Part 4.1–4.2	Voxel index를 환자의 physical space와 어떻게 연결하는가?	NIfTI affine, orientation, three-plane viewer

자료 기준	2026-09-06 repository snapshot · Part 1–3과 Lessons 4.1–4.2의 11개 notebook이 fresh project kernel에서 top-to-bottom 검증된 상태
--------------	---

박용민 · 경기대학교 AI컴퓨터공학부 컴퓨터공학전공

0. 전체 지도를 먼저 잡기

각 Part는 독립 과목이 아니다. Part 1이 출력과 정답의 계약, Part 2가 그 출력을 만드는 network, Part 3가 3D volume에서 그 network를 현실적으로 운용하는 방법, Part 4가 voxel array를 환자의 physical space와 연결하는 방법을 담당한다.

공통 notation

기호	의미	대표 예
B	batch size	한 step에 처리하는 patient patch 수
C	input channel	CT는 보통 C=1
K	output class 수	background 포함 10 classes라면 K=10
D, H, W	depth, height, width	3D spatial axes
pD, pH, pW	patch spatial size	예: 8×16×16

Training data flow

CT patch [B,C,D,H,W]
 → 3D network
 → logits [B,K,D,H,W]
 + target [B,D,H,W]
 → voxel-wise loss → backward → parameter update

Full-volume inference data flow

CT volume → overlapping patches → patch logits
 → global coordinates에 누적
 → count/weight normalization
 → full logits [B,K,D,H,W]
 → argmax(dim=1) → mask [B,D,H,W]

절대 혼동 금지

C는 입력 modality/channel, K는 모델이 경쟁시키는 class axis다. D/H/W는 지워야 할 class axis가 아니라 보존해야 할 spatial axes다.

Part 1. Segmentation Fundamentals

1.1 Segmentation Tensor Contract

Multi-class segmentation은 각 voxel마다 K개 class score를 출력하는 dense classification이다. 따라서 한 volume에 대한 예측은 scalar 하나가 아니라 spatial grid 전체의 class decision이다.

객체	Shape	Dtype	의미
CT input	[B,C,D,H,W]	float32/float16	모델 입력 intensity
Logits	[B,K,D,H,W]	floating	softmax 전 class score
Target	[B,D,H,W]	long	voxel마다 하나의 정답 class ID
Prediction	[B,D,H,W]	long	가장 큰 logit의 class ID

```
prediction[b,d,h,w] = argmax_k(logits[b,k,d,h,w])
```

`argmax(dim=1)`이 K axis만 제거하므로 `[B,K,D,H,W] → [B,D,H,W]`가 된다. `dim=2`를 사용하면 class axis K가 남고 depth D가 사라져 target과 비교할 수 없는 잘못된 mask가 된다.

Repository에서 확인한 값

```
Input      [2, 1, 32, 64, 64]
Logits     [2,10, 32, 64, 64]
Target     [2, 32, 64, 64] torch.int64
Prediction [2, 32, 64, 64] torch.int64
```

- Target에 K가 없는 이유: 정답은 voxel마다 이미 선택된 class ID 하나이기 때문.
- Logits는 확률이 아니다. softmax 전의 제한 없는 실수 score이며 합이 1일 필요가 없다.
- Background도 하나의 class이므로 K에 포함. 예: 9개 장기 + background = K=10.
- Contract validation은 shape/dtype 오류를 training 이전에 잡는 가장 싼 test.

1.2 Stable Softmax와 Voxel-wise Cross-Entropy

각 voxel의 K개 logits를 class probability로 바꾸되, 큰 값의 exponential overflow를 피하기 위해 최대 logit을 먼저 뺀다. 이 이동은 모든 class에 같은 상수를 빼므로 probability ratio를 바꾸지 않는다.

$$p_k = \exp(z_k - m) / \sum_j (\exp(z_j - m)), \quad m = \max_j (z_j)$$

입력 logits	Naive softmax	Stable softmax
[1000,1001,1002]	[nan,nan,nan]	[0.0900,0.2447,0.6652]

Cross-Entropy의 의미

$$L_{\text{voxel}} = -\log(p_{\text{target}}), \quad L = \text{mean}(L_{\text{voxel}})$$

정답 class의 probability가 1에 가까우면 loss가 0에 가까워지고, 정답 class에 낮은 probability를 주면 큰 penalty를 받는다. 구현에서는 class axis K에서 target ID에 해당하는 probability만 gather한 뒤 negative log를 취한다.

```
logits [B,K,D,H,W]
→ softmax(dim=1)
→ target.unsqueeze(1) [B,1,D,H,W]
→ gather(dim=1) → p_target [B,1,D,H,W]
→ -log → mean → scalar loss
```

Scratch 검증 직접 구현 loss 0.9076059461, PyTorch loss 0.9076058865, 절대 차이 약 5.96e-8. 수치 오차 범위에서 동일.

Multi-class와 Multi-label

구분	Multi-class	Multi-label
관계	한 voxel에서 classes가 상호 배타적	여러 class가 동시에 양성 가능
출력	softmax over K	class별 sigmoid
Target	class ID [B,D,H,W]	multi-hot [B,K,D,H,W]
대표 loss	CrossEntropyLoss	BCEWithLogitsLoss

1.3 Dice, IoU와 Empty-Mask Policy

Accuracy는 background가 압도적으로 많을 때 장기를 하나도 찾지 못한 모델도 높게 평가할 수 있다. 의료영상 segmentation은 foreground overlap을 직접 보는 Dice와 IoU가 핵심이다.

$$\text{Dice} = 2 \cdot \text{TP} / (2 \cdot \text{TP} + \text{FP} + \text{FN}), \text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

용어	조건	해석
TP	prediction=1, target=1	맞게 찾은 organ voxel
FP	prediction=1, target=0	없는 organ을 있다고 예측
FN	prediction=0, target=1	실제 organ을 놓침
TN	prediction=0, target=0	background를 맞힘; Dice 분자에는 없음

대표 test case

상황	TP/FP/FN	Dice	IoU
Perfect	모두 일치	1.0	1.0
Disjoint	TP=0	0.0	0.0
Partial	TP=1, FP=1, FN=1	0.5	0.3333
실습 예	TP=2, FP=1, FN=1	0.6667	0.5

Empty mask는 수학이 아니라 reporting policy

- Target empty + prediction empty: 보통 perfect 또는 평가 제외 중 하나를 protocol에서 사전 고정.
- Target empty + prediction non-empty: false-positive-only이므로 0점.
- Target non-empty + prediction empty: missed organ이므로 0점.
- Class별 score를 낸 뒤 patient/case macro 평균을 권장. voxel을 모두 합친 micro score는 큰 장기가 결과를 지배할 수 있음.

논문 방어 포인트

Empty-reference 처리, background 포함 여부, class/case aggregation을 결과를 본 뒤 바꾸면 안 된다. Experiment protocol에 먼저 freeze.

Part 1 Checkpoint · 반드시 말로 설명할 것

- `[B,K,D,H,W]`에서 K와 D의 의미가 왜 완전히 다른가?
- Target에 channel axis K가 없는 이유와 target dtype이 long인 이유는?
- 왜 `argmax(dim=1)`은 inference decision이고 training loss 안에서는 사용하지 않는가?
- Maximum subtraction이 softmax 결과를 바꾸지 않으면서 overflow를 막는 이유는?
- Background accuracy가 99%여도 organ segmentation 실패일 수 있는 이유는?
- Empty-empty case의 점수는 왜 반드시 protocol에 명시해야 하는가?

빠른 debugging 순서

1. input/logits/target/prediction shape 출력
2. dtype와 class ID 범위 확인: $0 \leq \text{target} < K$
3. softmax probability sum을 class axis에서 확인
4. scratch CE와 framework CE 비교
5. perfect/disjoint/partial/empty mask unit test
6. background와 class별 metric을 분리

통과 기준

Shape를 외우는 데서 끝나지 않고, 각 axis가 어떤 object를 index하는지 말할 수 있어야 한다.

Part 2. U-Net from Scratch

2.1 Convolution Shape와 Receptive Field

Convolution layer를 조립하기 전에 output spatial size를 계산할 수 있어야 skip concatenation 오류를 예방할 수 있다.

$$N_{out} = \text{floor}((N_{in} + 2 * P - \text{dilation} * (K - 1) - 1) / \text{stride}) + 1$$

설정	Output size	핵심
K=3, S=1, P=1	N 유지	same-size convolution
K=3, S=2, P=1, N=32	16	downsampling
K=5, S=1, P=0, N=32	28	valid convolution

Effective kernel과 receptive field

$$K_{effective} = \text{dilation} * (K - 1) + 1$$

$$\text{jump}_l = \text{jump}_{(l-1)} * \text{stride}_l, \text{RF}_l = \text{RF}_{(l-1)} + (K_{eff} - 1) * \text{jump}_{(l-1)}$$

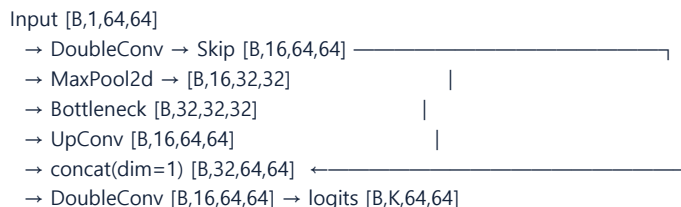
실습 trace는 receptive field $3 \rightarrow 5 \rightarrow 6 \rightarrow 10$, jump $1 \rightarrow 1 \rightarrow 2 \rightarrow 2$ 였다. Dilation=2인 3×3 convolution은 5×5 범위를 보지만 실제 sampling point는 9개뿐이다. Gradient mask로 span 5×5 와 sampled input 9개를 확인했다.

핵심 구분

Receptive-field span이 넓다는 것과 모든 위치를 조밀하게 사용한다는 것은 다르다. Dilation은 사이를 건너뛰며 넓게 본다.

2.2 Encoder–Decoder와 Skip Connection

Encoder는 context를 넓히고 memory를 줄이는 대신 fine boundary 위치를 잃는다. Decoder는 resolution을 회복하지만 pooling 이전의 정확한 위치 정보는 스스로 완벽히 되살릴 수 없다. Skip connection이 이 정보를 직접 전달한다.



Block	Spatial 변화	Channel 변화	역할
DoubleConv	유지	in → out	local feature extraction
Encoder pool	H,W 절반	유지	context 확대, memory 감소
Up-convolution	H,W 두 배	감소 가능	resolution 복원
Skip concat	반드시 동일	두 Tensor의 합	fine detail + context 결합
1×1 head	유지	features → K	class logits 생성

Concatenation 전 contract

- Batch size 동일.
- Spatial H/W 또는 D/H/W 동일. 홀수 크기와 padding policy가 다르면 crop/pad 필요.
- Channel은 같을 필요 없음. `dim=1`에서 이어 붙이므로 합산됨.
- Skip은 max-pooled Tensor가 아니라 pooling 이전의 high-resolution features.

2.3 Minimal 2D U-Net과 Tiny Overfit

Tiny overfit은 일반화 실험이 아니라 pipeline integrity test다. 1–2개 sample조차 외우지 못한다면 dataset, model, loss, optimizer 또는 metric 중 어딘가가 깨졌을 가능성이 높다.

실습 setup과 결과

항목	값
Input	[2,1,64,64], float32, CUDA
Target	[2,64,64], int64, 3 classes
Trainable parameters	117,107
Loss	1.018677 → 0.000688 over 200 steps
Per-class Dice	background/circle/rectangle 모두 1.0
Pixel accuracy	1.0

무엇을 증명하고 무엇을 증명하지 않는가

증명 가능	증명 불가능
forward/backward가 연결됨	새 환자에 대한 generalization
target와 logits contract가 맞음	임상적 유용성
optimizer가 해당 sample을 학습 가능	domain shift robustness
metric implementation이 완벽 예측에서 1.0	test-set 성능

Tiny overfit 실패 진단

- Loss가 전혀 감소하지 않음: learning rate, detached graph, optimizer parameter 등록 확인.
- Accuracy만 높고 organ Dice가 0: background collapse와 class imbalance 확인.
- Loss는 감소하지만 metric 고정: argmax axis, label mapping, metric class ID 확인.
- 출력 shape 불일치: encoder–decoder spatial 계산과 final head channel K 확인.

Part 2 Checkpoint · U-Net을 그림 없이 추적하기

필수 설명

- 왜 pooling은 receptive field와 context를 늘리지만 boundary localization을 희생하는가?
- Skip feature와 pooled feature는 각각 어떤 shape이며 왜 둘 다 필요한가?
- Decoder에서 upsample 후 skip과 `dim=1`로 concatenate하는 이유는?
- Final 1×1 convolution이 spatial size를 유지하면서 K logits를 만드는 원리는?
- Tiny overfit Dice 1.0을 얻어도 논문 성능 주장으로 사용할 수 없는 이유는?

One-level shape drill

문제: [B=2,C=1,H=64,W=64]

DoubleConv out=16 → Pool(2) → Bottleneck out=32

→ UpConv out=16 → Skip concat → DoubleConv out=16 → Head K=3

정답:

skip [2,16,64,64], pooled [2,16,32,32]

bottleneck [2,32,32,32], up [2,16,64,64]

concat [2,32,64,64], logits [2,3,64,64]

prediction [2,64,64]

연구 습관

Network diagram의 화살표만 보지 말고 모든 node 옆에 Tensor shape를 적는다. 대부분의 구현 오류는 shape trace에서 먼저 드러난다.

Part 3. Volumetric Learning and Patch Mechanics

3.1 Conv3D Tensor Flow와 Memory

Conv3D는 slice 하나가 아니라 depth까지 동시에 처리하므로 3D context를 학습하지만 activation memory가 급격히 증가한다. 8 GB GPU에서는 architecture보다 먼저 patch size와 batch size의 현실성을 계산해야 한다.

$$\text{Raw tensor memory [bytes]} = \text{number_of_elements} * \text{bytes_per_element}$$

Tensor	FP32	FP16	변화
[1,32,32,64,64]	16 MiB	8 MiB	base
Batch ×2	32 MiB	16 MiB	2×
Depth ×2	32 MiB	16 MiB	2×
H와 W ×2	64 MiB	32 MiB	4×
D,H,W 모두 ×2	128 MiB	64 MiB	8×

실습에서 `[1,8,32,64,64]` 3D output은 4.0 MiB였고 대응 2D output보다 32배 컸다. 이는 depth=32만큼 spatial element가 추가됐기 때문이다.

Raw output memory ≠ training peak

항목	실습 측정	의미
Returned outputs	4.5 MiB	skip + pooled Tensor 자체
Additional training peak	34.0 MiB	forward 저장 + gradient + workspace 등
Peak allocated	34.5 MiB	PyTorch가 실제 Tensor에 할당
Peak reserved	46.0 MiB	CUDA allocator가 확보한 pool

용량 판단

Parameter memory만 보고 학습 가능하다고 결론 내리지 말 것. Activation, backward graph, optimizer state, temporary workspace와 allocator reserve를 함께 측정.

3.2 Crop, Padding과 Patch Sampling

Full 3D volume을 GPU에 올릴 수 없으면 작은 patch를 학습 단위로 사용한다. 이때 crop 함수의 geometry와 patch center sampling policy가 실제 학습 분포를 결정한다.

Center-based crop

```
volume [B,C,D,H,W]
center = (cz,cy,cx), patch = (pD,pH,pW)
start = center - floor(patch/2)
end   = start + patch_size
volume 밖 영역은 먼저 계산한 뒤 pad
```

- 중앙 patch: `[1,1,8,10,12]`에서 center `(4,5,6)`, patch `(4,6,8)` → `[1,1,4,6,8]`.
- 경계 patch: center `(0,1,1)`에서 부족한 앞쪽 영역을 padding해 요청 shape 유지.
- Image padding과 label padding value는 의미가 다름. CT는 preprocessing convention, label은 background/ignore policy와 일치 필요.

Case selection과 patch-center selection

단계	질문	예
Case selection	어느 patient/volume을 뽑는가?	dataset sampler
Center selection	선택된 volume의 어디를 자르는가?	uniform 또는 foreground

Uniform vs foreground sampling

Sampler	Center 선택	Foreground hit (실습 1000회)	효과
Uniform	전체 voxel에서 균일	43/1000 = 4.3%	대부분 background
Foreground	양성 voxel 중 선택	1000/1000 = 100%	organ signal 보장

중요한 trade-off Foreground oversampling은 rare organ을 더 자주 보게 하지만 실제 prevalence를 바꾼다. 항상 foreground만 뽑는 것이 정답이 아니라 비율을 실험 변수로 통제해야 한다.

3.3 Sliding-Window Inference

Training은 patch 단위여도 평가는 full volume mask가 필요하다. Sliding-window inference는 겹치는 patch를 전체 volume에 다시 배치하고 같은 voxel의 prediction을 평균한다.

시작 좌표와 coverage

- Regular starts: `0, step, 2-step, ...`.
- 마지막 start가 `image\_size - patch\_size`가 아니면 해당 값을 추가해 끝 경계까지 coverage.
- 실습: volume `(20,32,40)`, patch `(8,16,16)`, step `(4,8,8)`.
- Starts: Z=4개, Y=3개, X=4개 → 총 $4 \times 3 \times 4 = 48$ patches.

Accumulation과 normalization

$$\text{final_logits}(v) = \text{sum_p}(\text{logits_p}(v)) / \text{count}(v)$$

accumulated [B,K,D,H,W] = 0

count_map [1,1,D,H,W] = 0

for patch_logits, (z,y,x):

 accumulated[..., region] += patch_logits

 count_map[..., region] += 1

full_logits = accumulated / count_map

prediction = full_logits.argmax(dim=1)

Identity reconstruction test

검사	결과	증명
Minimum count	1	미coverage voxel 없음
Maximum count	8	내부 voxel이 최대 8 patch에 포함
$13,460 \times 8$	107,680	누적 좌표가 정확
Normalize 후 reconstruction	max/mean error 0	조립 pipeline 정확

해석 한계

Identity patch를 완벽히 복원한 것은 inference 조립 코드가 맞다는 뜻이다. 실제 model accuracy나 새 환자 generalization을 증명하지 않는다.

Part 4. Medical Image Geometry and CT

4.1 NIfTI Array와 Affine

NIfTI의 voxel array는 intensity를 저장하지만 array index 자체는 환자의 해부학적 위치가 아니다. 4×4 affine이 voxel index `(i,j,k)`를 millimeter 단위 physical coordinate `(x,y,z)`로 연결한다.

$$[x, y, z, 1]^T = \text{affine} * [i, j, k, 1]^T$$

실습 geometry

array shape (I,J,K) = (6,8,10)
 voxel spacing (2.0,1.5,3.0) mm
 origin XYZ (-120,-90,-60) mm
 axis codes ('R','A','S')

affine =
 [[2.0, 0.0, 0.0, -120.0],
 [0.0, 1.5, 0.0, -90.0],
 [0.0, 0.0, 3.0, -60.0],
 [0.0, 0.0, 0.0, 1.0]]

Index-to-physical 계산

$$\text{index } (2, 3, 4) \rightarrow (-120 + 2 * 2, -90 + 3 * 1.5, -60 + 4 * 3)$$

$$\text{physical XYZ} = (-116.0, -85.5, -48.0) \text{ mm}$$

객체	좌표계	단위	질문
Array index	IJK	voxel	배열에서 몇 번째 원소인가?
Physical coordinate	XYZ	mm	환자 공간의 어디인가?
Affine	IJK → XYZ	mm/voxel + 방향 + origin	두 좌표계를 어떻게 연결하는가?

검증 결과 세 voxel의 scratch affine 계산이 nibabel 결과와 일치. Inverse affine round trip의 maximum index error도 0.0.

Continuous index를 버리지 말 것

Physical coordinate가 항상 voxel center에 놓이는 것은 아니다. 실습의 `(-115,-85.5,-48) mm`는 continuous index `(2.5,3,4)`로 복원됐다. Resampling과 interpolation에서는 반올림 전 continuous coordinate가 필요하다.

Geometry contract Shape만 같다고 image와 label이 정렬된 것은 아니다. Shape, spacing, affine, orientation을 함께 검사해야 같은 physical anatomy를 가리킨다고 말할 수 있다.

4.2 Orientation과 Three-Plane Viewer

Array axis 번호와 해부학적 방향은 동일한 개념이 아니다. `nib.aff2axcodes(affine)`가 각 voxel axis의 증가 방향을 R/L, A/P, S/I code로 알려준다. 실습 volume은 RAS orientation이었다.

Plane	고정 index	추출 후 Shape	표시 전 transpose
Sagittal	I	[J,K] = [50,60]	[K,J] = [60,50]
Coronal	J	[I,K] = [40,60]	[K,I] = [60,40]
Axial	K	[I,J] = [40,50]	[J,I] = [50,40]

```
volume[I,J,K]
sagittal = volume[i, :, :] # [J,K]
coronal  = volume[:, j, :] # [I,K]
axial    = volume[:, :, k] # [I,J]
```

```
display: transpose + origin='lower' + physical extent
```

Transpose는 저장된 voxel을 재배열해 화면의 세로·가로 axis에 맞추는 표시 단계다. Orientation 정보 없이 무조건 flip하거나 transpose하면 좌우가 뒤집힌 것처럼 보일 수 있으므로 axis code와 affine을 먼저 확인한다.

RAS와 LPS에서 index가 달라도 같은 점일 수 있음

표현	Landmark index	Intensity	Physical XYZ
RAS	[33,25,30]	300	[26,0,0] mm
LPS	[6,24,30]	300	[26,0,0] mm

RAS index != LPS index, but RAS world XYZ = LPS world XYZ

I/J array를 뒤집으면 landmark index는 달라진다. 동시에 affine을 갱신하면 두 index가 같은 환자 위치를 가리킨다. 따라서 index equality가 아니라 affine을 적용한 physical-coordinate equality로 geometry 보존을 검증한다.

Canonical RAS 검증 LPS volume을 `nib.as\_closest\_canonical`로 RAS 복원한 결과 axis code RAS, array 일치, affine 일치, maximum voxel difference 0.0.

의료영상 안전 규칙 좌우 laterality가 있는 organ은 viewer 모양만 보고 판단하지 않는다. Orientation code, affine, landmark physical coordinate를 함께 확인한다.

Part 4.1–4.2 Checkpoint · Geometry를 말로 추적하기

- Voxel index `(i,j,k)`와 physical coordinate `(x,y,z)`는 무엇이 다른가?
- Affine의 diagonal과 마지막 column은 axis-aligned 예제에서 각각 무엇을 뜻하는가?
- Index `(2,3,4)`가 `(-116,-85.5,-48) mm`로 변환되는 계산을 직접 적을 수 있는가?
- Inverse affine 결과가 integer가 아닌 continuous index일 수 있는 이유는?
- Sagittal/coronal/axial slice에서 고정하는 array axis와 남는 Shape는 무엇인가?
- Slice display 전에 transpose가 필요한 이유와 무조건 flip하면 안 되는 이유는?
- RAS와 LPS landmark index가 달라도 physical point가 같을 수 있는 이유는?
- Canonical reorientation 뒤 무엇을 비교해야 geometry가 보존됐다고 말할 수 있는가?

Geometry debugging ladder

1. array shape와 dtype 확인
2. header spacing 확인
3. full 4x4 affine 확인
4. orientation axis codes 확인
5. known landmark: index -> physical XYZ 확인
6. image와 label의 geometry contract 비교
7. reorientation/resampling 후 physical equality 검증
8. axial/coronal/sagittal overlay 시각 검사

현재 경계

4.1–4.2는 geometry를 읽고 방향을 검증하는 단계까지 완료. Voxel spacing을 실제로 바꾸는 interpolation policy는 다음 Lesson 4.3에서 학습.

Parts 1–4.2 통합 · 하나의 Geometry-Aware 3D System

단계	Tensor/Data	지켜야 할 invariant	실패 징후
Patch sampling	CT/label patch	동일 좌표, 동일 spatial shape	image-label misalignment
Geometry load	array + affine	spacing/orientation/physical alignment	좌우 반전, shifted label
Network	[B,C,pD,pH,pW]	logits channel=K	head channel 오류
Training loss	logits + [B,pD,pH,pW]	target long, class ID 범위	CE shape/dtype error
Patch inference	patch logits	global origin 보존	shifted prediction
Accumulation	sum + count map	모든 count > 0	NaN/holes
Decision	full logits	argmax over K only	spatial axis 소실
Evaluation	mask + target	frozen empty/aggregation policy	score 재현 불가

실전 debugging ladder

- Level 1 Tensor contract unit tests
- Level 2 Stable loss + metric test cases
- Level 3 Single block forward/backward
- Level 4 Tiny overfit on 1–2 samples
- Level 5 Patch crop/padding visualization
- Level 6 Identity sliding-window reconstruction
- Level 7 Validation cohort evaluation
- Level 8 Locked test only after method freeze

아래 level이 실패한 상태에서 위 level의 성능을 해석하면 원인 분리가 불가능하다. OLES3D 연구는 이 ladder를 고정해 method 변화와 implementation bug를 분리한다.

Memory-aware design rule

$$(2*D)*(2*H)*(2*W) = 8*(D*H*W) \Rightarrow \text{activation elements} \sim 8x$$

- OOM이면 무조건 backbone부터 줄이지 말고 batch, patch, channels, precision, checkpointing 순서로 trade-off 기록.
- Patch 크기를 바꾸면 memory뿐 아니라 organ context와 foreground hit probability도 변함.
- Training patch policy와 inference sliding-window policy는 목적이 다르므로 별도 parameter로 관리.

자가시험 · 답을 가리고 직접 설명하기

1. `[2,10,32,64,64]` logits에서 target과 prediction shape는 무엇인가?
2. `argmax(dim=2)`가 잘못된 이유를 axis 의미로 설명하라.
3. Stable softmax에서 maximum logit을 빼는 이유와 결과가 보존되는 이유는?
4. TP=2, FP=1, FN=1일 때 Dice와 IoU를 계산하라.
5. Background-only prediction의 accuracy가 높아도 실패인 이유는?
6. `[1,16,32,64,64]` FP32 Tensor의 raw memory는 몇 MiB인가?
7. D, H, W를 모두 2배로 만들면 activation element 수는 몇 배인가?
8. U-Net skip connection이 전달하는 정보와 concatenate 조건은?
9. Tiny overfit Dice 1.0으로 새 환자 성능을 결론 낼 수 없는 이유는?
10. Uniform sampling과 foreground sampling의 center 선택 차이는?
11. Sliding-window에서 count map이 필요한 이유는?
12. Identity reconstruction error 0이 증명하는 것과 증명하지 않는 것은?
13. Training target에는 K가 없지만 model logits에는 K가 있는 이유는?
14. Raw Tensor memory와 GPU peak reserved memory가 다른 이유는?
15. Voxel index `(i,j,k)`와 physical coordinate `(x,y,z)`의 차이는?
16. Index `(2,3,4)`, spacing `(2,1.5,3) mm`, origin `(-120,-90,-60) mm`의 physical coordinate는?
17. Sagittal, coronal, axial slice에서 각각 고정하는 array axis는?
18. RAS와 LPS에서 landmark index가 달라도 같은 anatomy를 가리킬 수 있는 이유는?
19. Canonical RAS 변환 후 geometry 보존을 검증할 항목은?
20. Image와 label의 Shape만 같아도 정렬됐다고 결론 낼 수 없는 이유는?

통과 방법

숫자만 맞이지 말고 Tensor shape와 data flow를 그림으로 적은 뒤, 잘못된 구현이 어떤 output을 만드는지까지 설명.

자가시험 정답과 마지막 Cheat Sheet

1. Target `[2,32,64,64]`, prediction `[2,32,64,64]`.
2. dim=2는 D를 제거해 `[B,K,H,W]`를 만들고 class axis K를 남김. 제거 대상은 dim=1의 K.
3. exp overflow 방지. 모든 logit에 같은 상수를 빼므로 softmax의 상대 비율과 최종 probability는 동일.
4. Dice=4/6=0.6667, IoU=2/4=0.5.
5. TN이 압도해 accuracy를 지배하지만 organ TP는 0이고 FN은 클 수 있음.
6. $1 \times 16 \times 32 \times 64 \times 64 \times 4 \text{ bytes} = 8,388,608 \text{ bytes} = 8 \text{ MiB}$.
7. $2 \times 2 \times 2 = 8$ 배.
8. Pooling 이전의 high-resolution localization 정보. Batch/spatial shape가 같아야 하고 channel은 합산됨.
9. Training sample 암기 test일 뿐 unseen patient distribution을 평가하지 않았기 때문.
10. Uniform은 전체 voxel에서 center 선택, foreground는 양성 class voxel 중 center 선택.
11. 같은 global voxel에 여러 patch logits가 더해진 횟수로 나눠 평균하기 위해 필요.
12. 좌표·누적·정규화 pipeline의 정확성은 증명. Model accuracy/generalization은 증명하지 않음.
13. Logits는 K개 후보 score, target은 그중 정답 class ID 하나.
14. Training은 saved activation, gradient, optimizer/workspace와 CUDA allocator reserve까지 필요.
15. IJK는 array element 위치이고 XYZ는 affine으로 변환한 환자 공간의 millimeter 위치.
16. $(-116, -85.5, -48) \text{ mm}$. 각 axis에서 $\text{origin} + \text{index} \times \text{spacing}$ 계산.
17. Sagittal은 I, coronal은 J, axial은 K를 고정.
18. Array를 flip/reorder하면서 affine도 함께 갱신하면 서로 다른 index가 동일한 physical XYZ를 가리킬 수 있음.
19. Axis code와 함께 landmark physical coordinate, canonical array, affine 및 voxel difference 비교.
20. 서로 다른 spacing, origin, direction/orientation의 array가 우연히 같은 Shape일 수 있기 때문.

최종 8줄 요약

- Segmentation = voxel마다 수행하는 K-class classification.
- Training contract = logits `[B,K,D,H,W]` + target `[B,D,H,W]`.
- U-Net = context를 얻는 encoder + resolution을 복원하는 decoder + localization을 전달하는 skip.
- Tiny overfit = generalization test가 아니라 pipeline integrity test.
- 3D memory는 D·H·W에 비례하므로 patch training이 현실적인 기본 단위.
- Full-volume inference = overlapping logits 누적 ÷ count/weight map → argmax over K.
- NIfTI geometry = voxel array + 4×4 affine; index와 patient-space millimeter를 분리.
- Orientation 변경은 index를 바꾸지만 affine을 함께 갱신하면 physical anatomy는 보존.

다음 Part

다음 Lesson 4.3에서는 spacing을 바꿀 때 output Shape와 affine을 함께 갱신하고, CT image에는 trilinear, segmentation label에는 nearest-neighbor interpolation을 적용하는 이유를 검증한다.

Source notebooks: [studies/prerequisites/part01_segmentation_fundamentals](#) · [part02_unet_from_scratch](#) · [part03_volumetric_learning](#) · [part04_medical_image_geometry_ct/01–02](#)